

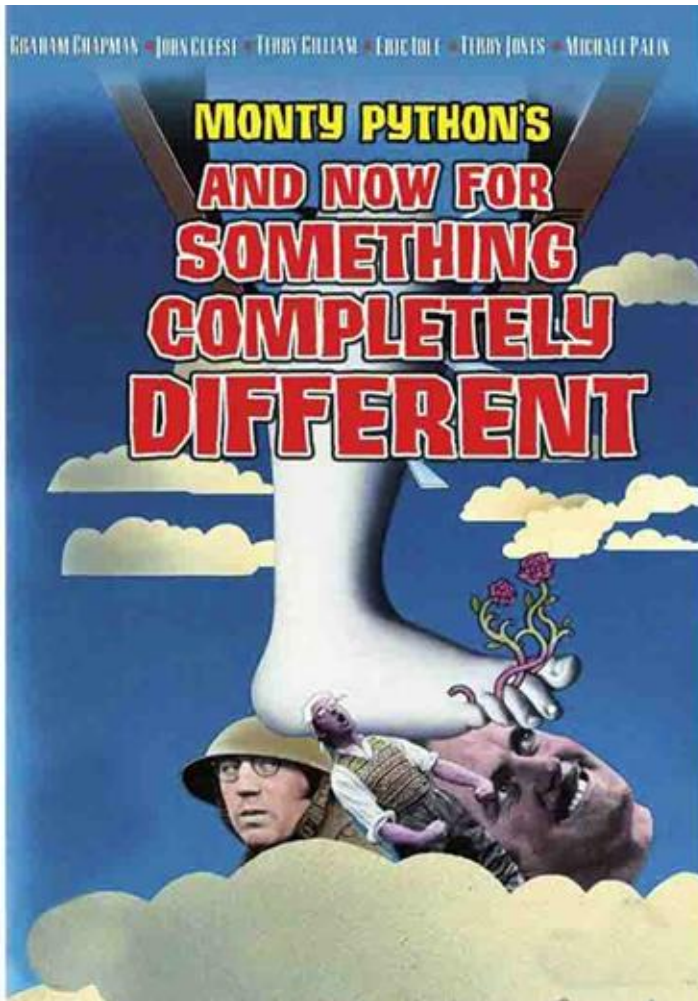
Divide and Conquer Part 2

We have divided the divide-and-conquer topic into two lectures, and now we'll conquer them!



Reminders

- Homework 2 goes out (late) tonight
 - Due next week 9/16 at 8pm
 - Accepted until 9:59pm with 5% penalty
 - Lowest 2 homework grades are dropped
 - (this is for illness, power outage, etc.)



A Design
Technique:
Divide and
Conquer

Overview: Divide-and-Conquer Algorithms

Main Idea:

1. **Divide** the input into smaller sub-problems
2. Solve each sub-problem *recursively*
3. **Conquer**: Combine the sub-solutions to get a solution for the original input

Designing the Algorithm + Proving Correctness: an “art”

- Depends on problem structure, ad-hoc, creative

Running time Analysis: “mechanical”

- Express runtime using a recurrence
- Can often solve using the “Master Theorem”

Mergesort

Unsorted array of length n

Unsorted array of length $n/2$

Unsorted array of length $n/2$

Sorted array of length $n/2$

Sorted array of length $n/2$



Recurrences and Running Times

$T(n)$ = worst case running time of mergesort on input of length n

$$T(n) = 2 T(n/2) + O(n)$$


Two recursive calls on problems of size $n/2$

Merge two arrays of size $n/2$

Solving Recurrences

The Master Theorem

Let $T(n)$ be the following recurrence, where $k \geq 1$, $b > 1$, $d \geq 0$, $w \geq 0$ are constants (they do not depend on n):

$$T(n) = kT(n/b) + O(n^d \log^w n)$$

You can use
this as a
black box

Then:

$$T(n) = \begin{cases} O(n^d \log^w n) & \text{if } k < b^d \\ O(n^d \log^{w+1} n) & \text{if } k = b^d \\ O(n^{\log_b k}) & \text{if } k > b^d \end{cases}$$



For Mergesort: $k=2$, $b=2$, $d=1$, $w=0 \Rightarrow O(n \log n)$.

Another example of divide and conquer:

Integer Multiplication

Input: Two n -digit positive integers x, y

Output: The product $x \cdot y$

“Primitive operations” that can be done in constant time:

- add or multiply two single-digit numbers
- “shift” a number (i.e. add a 0 to the end)

The Grade-School Algorithm

		2	0	3
	×	3	7	6
	1	2	1	8
1	4	2	1	0
6	0	9	0	0
7	6	3	2	8

What is the running time?

$O(n^2)$



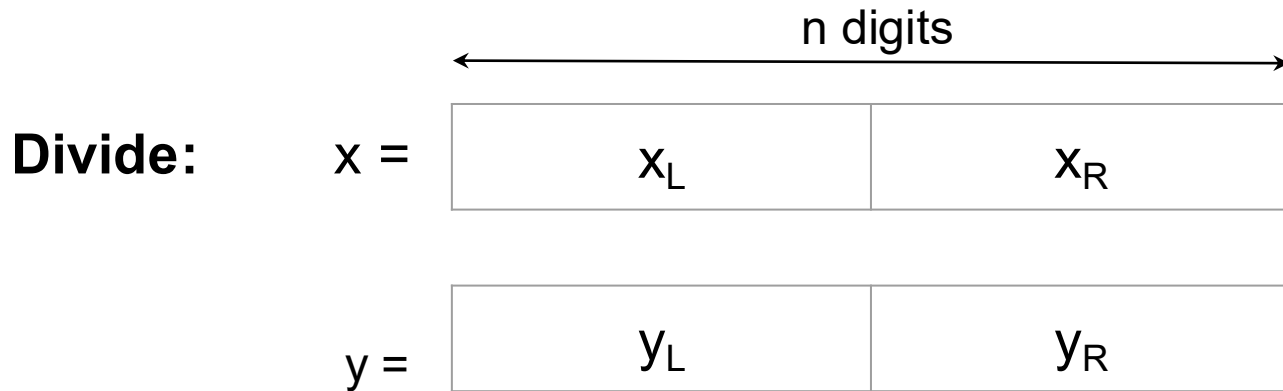
An algorithm designer's mantra

“Perhaps the most important principle for the good algorithm designer is to refuse to be content.”

- Aho, Hopcroft, Ullman, *The Design and Analysis of Computer Algorithms* (1974)

Divide-and-Conquer:

Integer Multiplication



Conquer: $x \cdot y = (x_L \cdot 10^{n/2} + x_R) \cdot (y_L \cdot 10^{n/2} + y_R)$

$$= x_L y_L \cdot 10^n + (x_L y_R + x_R y_L) \cdot 10^{n/2} + x_R y_R$$

Running Time:

$$T(n) = 4 T(n/2) + O(n)$$

Solving the recurrence

The Master Theorem

Let $T(n)$ be the following recurrence, where $k \geq 1$, $b > 1$, $d \geq 0$, $w \geq 0$ are constants (they do not depend on n):

$$T(n) = kT(n/b) + O(n^d \log^w n)$$

Then:

$$T(n) = \begin{cases} O(n^d \log^w n) & \text{if } k < b^d \\ O(n^d \log^{w+1} n) & \text{if } k = b^d \\ O(n^{\log_b k}) & \text{if } k > b^d \end{cases}$$

$$T(n) = 4 T(n/2) + O(n)$$

Conclusion: Naïve divide and conquer Algorithm takes $O(n^2)$ time.

(Earlier, Gauss used the same trick in a different context)

Karatsuba's idea!

$\Theta(n^2)$

Around 1956, the famous Soviet mathematician **Andrey Kolmogorov** conjectured that this is the *best possible way* to multiply two numbers together.

Just a few years later, Kolmogorov's conjecture was shown to be spectacularly wrong.

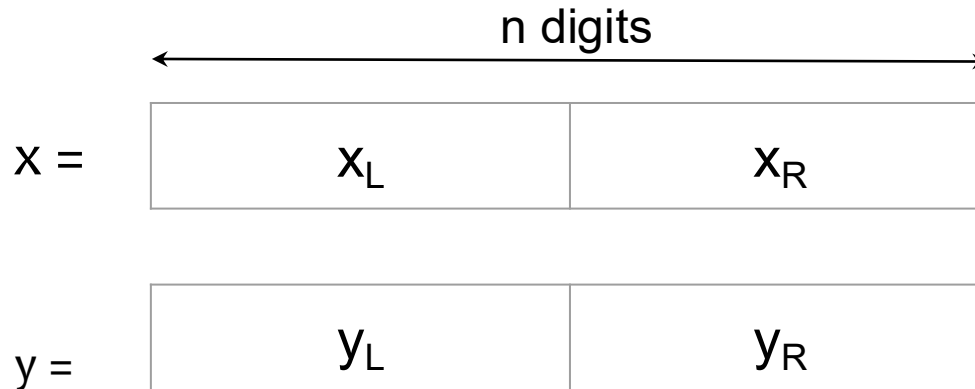
In 1960, Anatoly Karatsuba, a 23-year-old mathematics student in Russia, discovered a **sneaky algebraic trick** that reduces the number of multiplications needed.

We only need 3 recursive calls rather than 4!



Karatsuba's idea!

Divide:



sigh When I was in elementary school....



I could have saved a lot of time

Conquer:

$$x \cdot y = \boxed{x_L y_L} 10^n + \cancel{(x_L y_R + x_R y_L)} 10^{n/2} + \boxed{x_R y_R}$$

$\boxed{(x_L + x_R)(y_L + y_R) - x_L y_L - x_R y_R}$
 M_1

$$= M_2 \cdot 10^n + (M_1 - M_2 - M_3) \cdot 10^{n/2} + M_3$$

Next: Analyze the runtime of Karatsuba with the Master Theorem



Solving the recurrence

The Master Theorem

Let $T(n)$ be the following recurrence, where $k \geq 1$, $b > 1$, $d \geq 0$, $w \geq 0$ are constants (they do not depend on n):

$$T(n) = kT(n/b) + O(n^d \log^w n)$$

Then:

$$T(n) = \begin{cases} O(n^d \log^w n) & \text{if } k < b^d \\ O(n^d \log^{w+1} n) & \text{if } k = b^d \\ O(n^{\log_b k}) & \text{if } k > b^d \end{cases}$$

$$T(n) = 3T(n/2) + O(n)$$

Conclusion: Karatsuba's Algorithm takes $O(n^{\log_2 3})$ time. $O(n^{1.585})$

Question: Is it possible to do even better than Karatsuba multiplication?

Answer: Yes - the best known result is $O(n \log n)$
[Harvey, van der Hoeven, 2019]

Unfortunately, the hidden constants are *enormous*:

“...the proof given in our paper only works for ludicrously large numbers. Even if each digit was written on a hydrogen atom, there would not be nearly enough room available in the observable universe to write them down.”

Open problem: Can this be improved to $O(n)$?

Conjecture: No (but we don't know—maybe possible!)

Another example of divide and conquer:

Closest Pair of Points

Closest Pair Data Structures: Applications

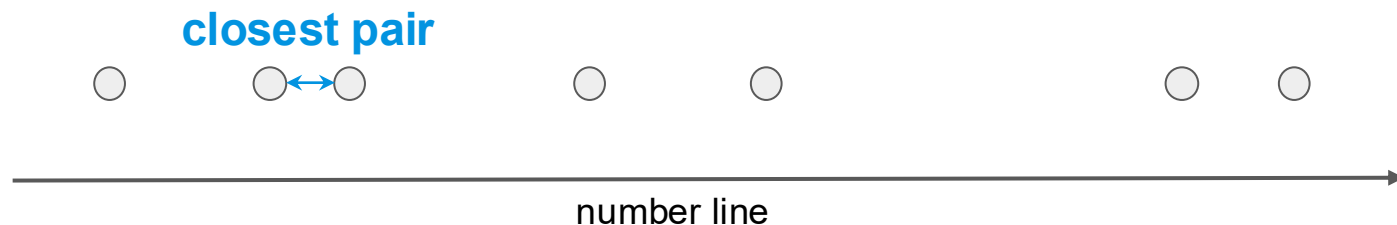
The following algorithms and applications can be implemented efficiently using our new closest pair data structures, or involve closest pair computation as important subroutines.

- Dynamic minimum spanning trees
- Two-optimization heuristics in combinatorial optimization
- [Straight skeletons and roof design](#)
- [Ray-intersection diagram](#)
- Other collision detection applications
- Hierarchical clustering
- Traveling salesman heuristics
- Greedy matching
- Constructive induction
- Gröbner bases

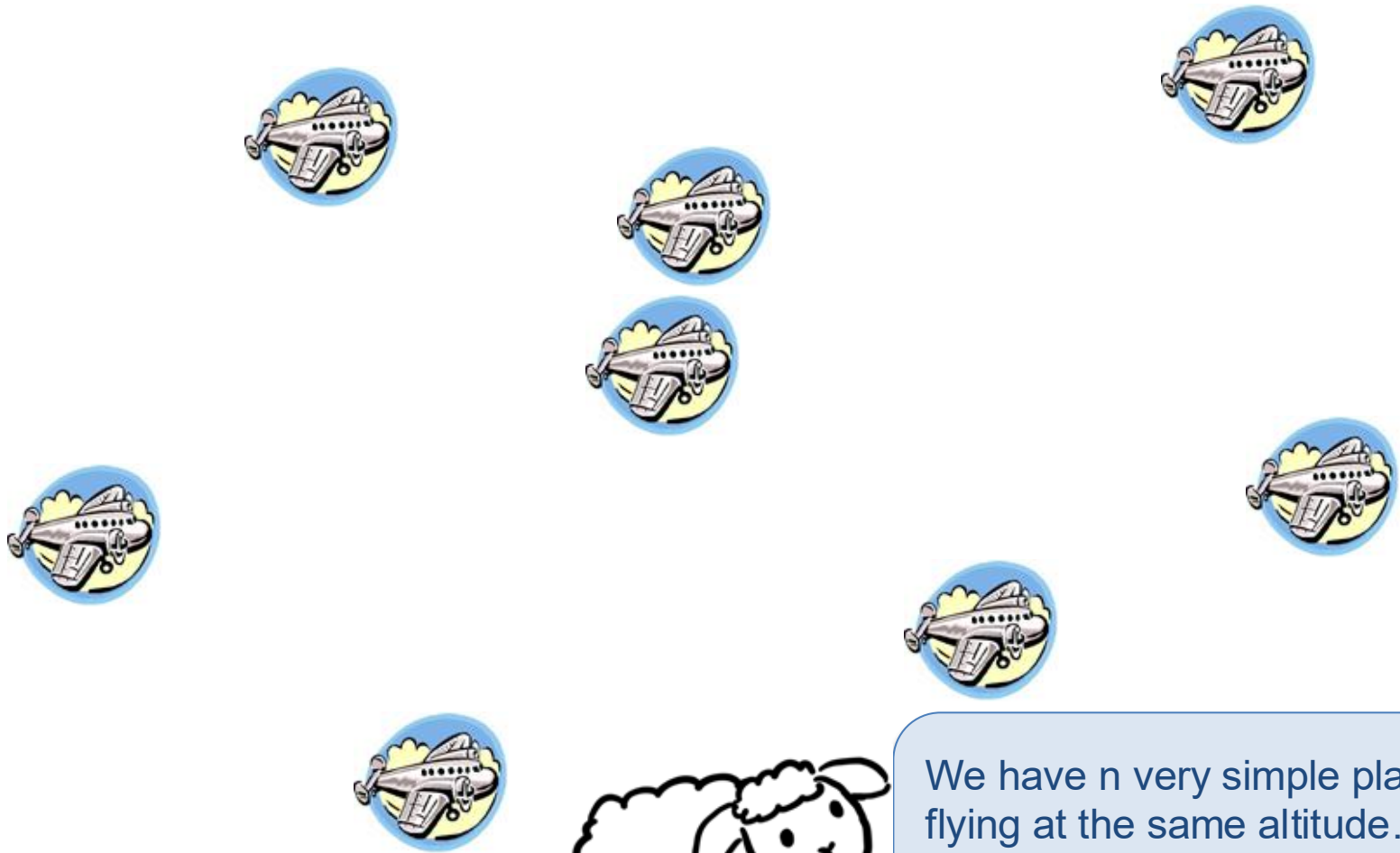
[David Eppstein](#), [Information & Computer Science](#), [UC Irvine](#), .

Warm-up: Closest Pair of Points in 1-D

- * **Problem:** Given n real numbers $x_1, x_2, \dots, x_n$, find $i \neq j$ with the smallest $|x_i - x_j|$.
- * **Solution:**
 - * Sort the points.
 - * Go over the list and compute the distances between the adjacent points.
 - * Return the pair of adjacent points with the minimal distance.
- * **Runtime:** $O(n \log n)$



Closest Pair of Points in 2-D



We have n very simple planes flying at the same altitude... I guess they are n plain planes in the plane!

Closest Pair of Points

Input: A list of n points in $\mathbb{R}^2$: $[(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)]$

Output: A closest pair of points

Goal: $O(n \log n)$ algorithm

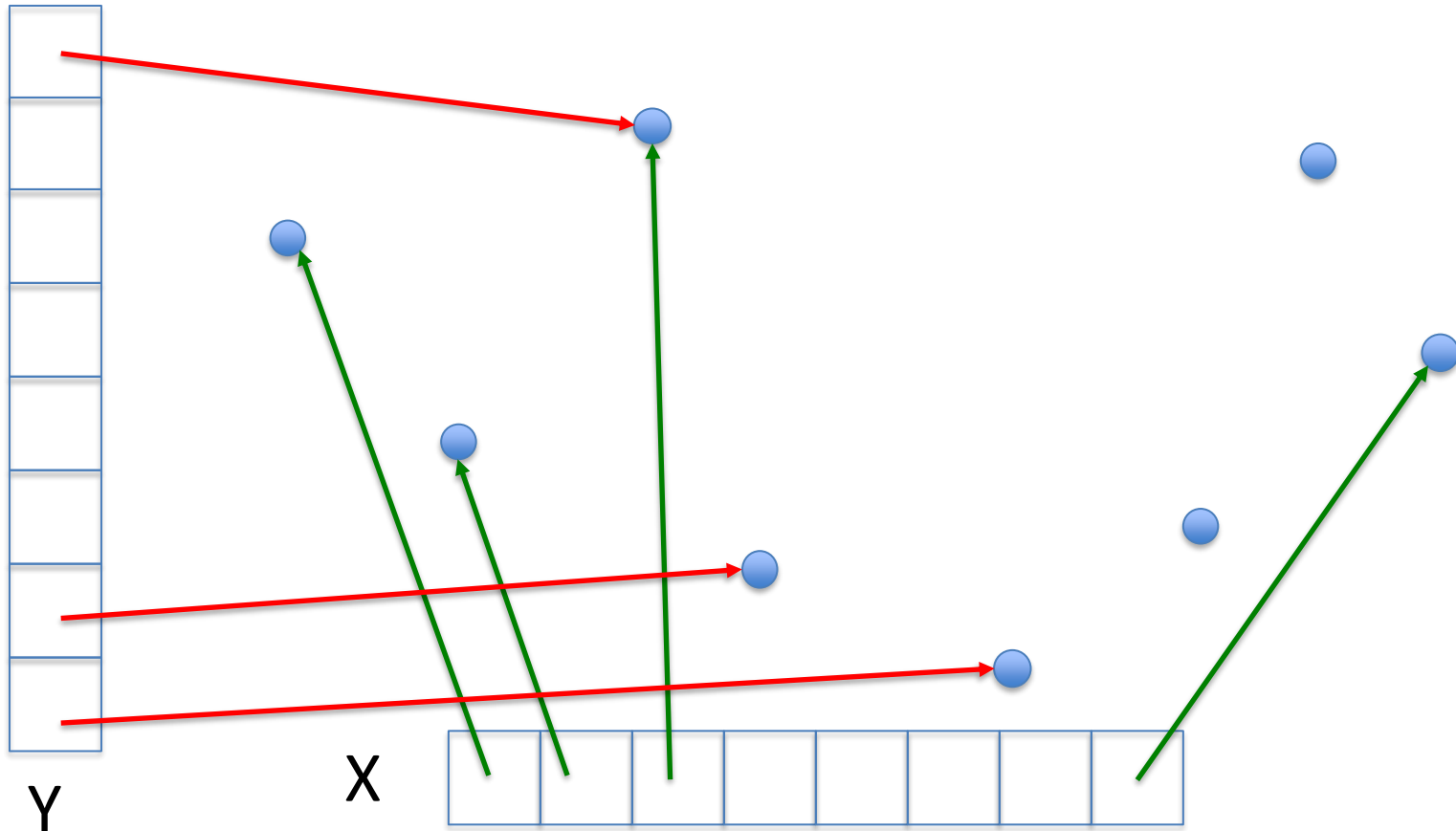
Working backwards: We want a “mergesort” recurrence relation!

$$T(n) = 2T(n/2) + O(n)$$

If we're willing to spend $O(n \log n)$ time, we may as well sort these points!



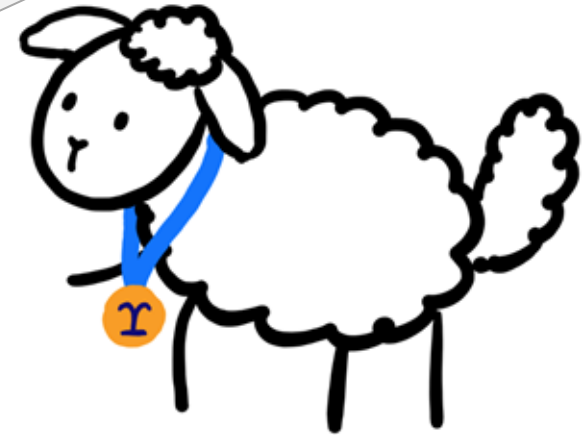
Build two copies of the input list: One sorted by x-coordinate and one sorted by y-coordinate



If we're willing to spend $O(n \log n)$ time, we may as well sort these points!



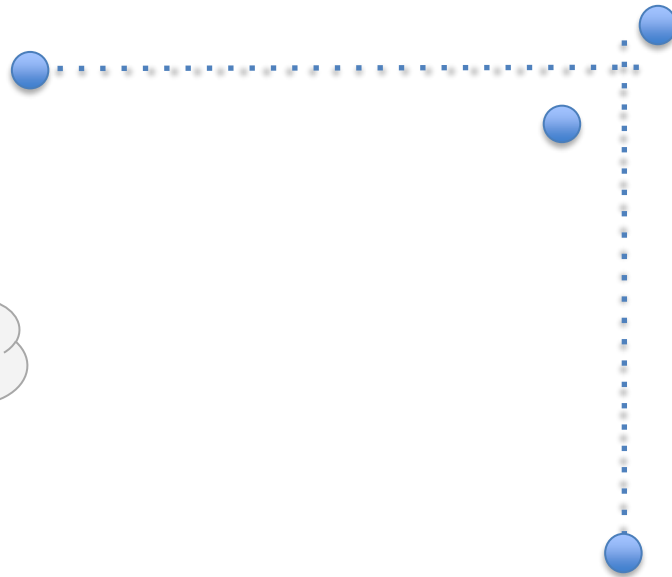
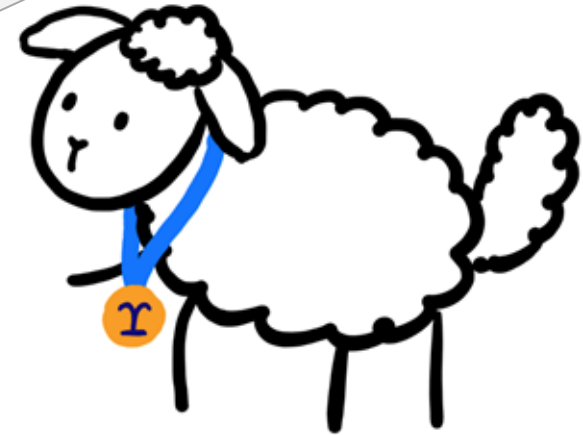
No need to use divide-and-conquer! Just run my new algorithm! Iterate through the sorted list X and compute the distance between adjacent points. Then do the same for the list Y. Then take the minimum distance we found!



Hmmm...



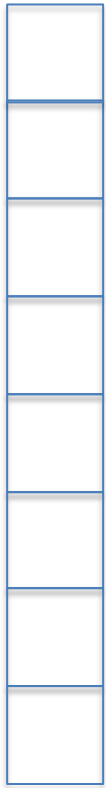
No need to use divide-and-conquer! Just run my new algorithm! Iterate through the sorted list X and compute the distance between adjacent points. Then do the same for the list Y. Then take the minimum distance we found!



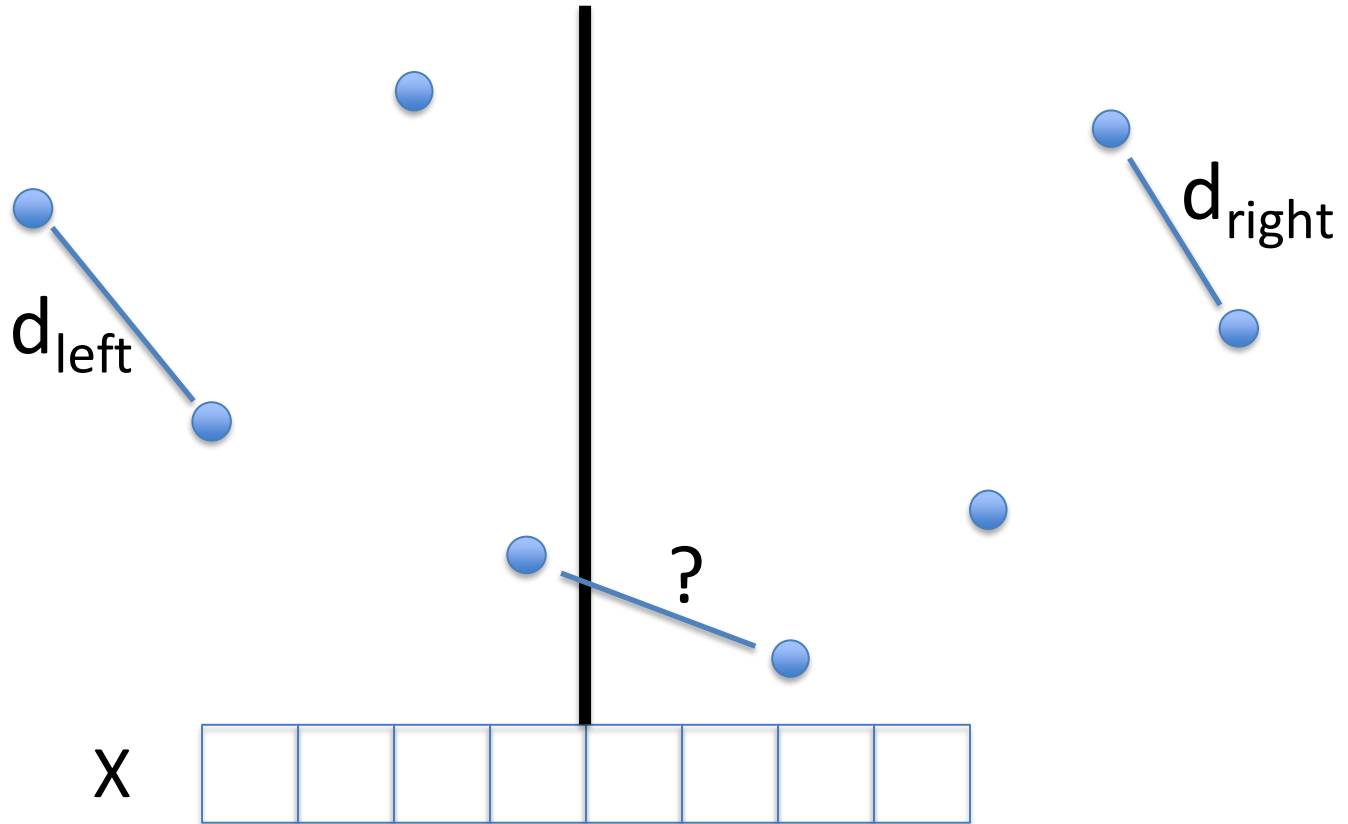
Counter-
example!



Divide-and-Conquer!



Y



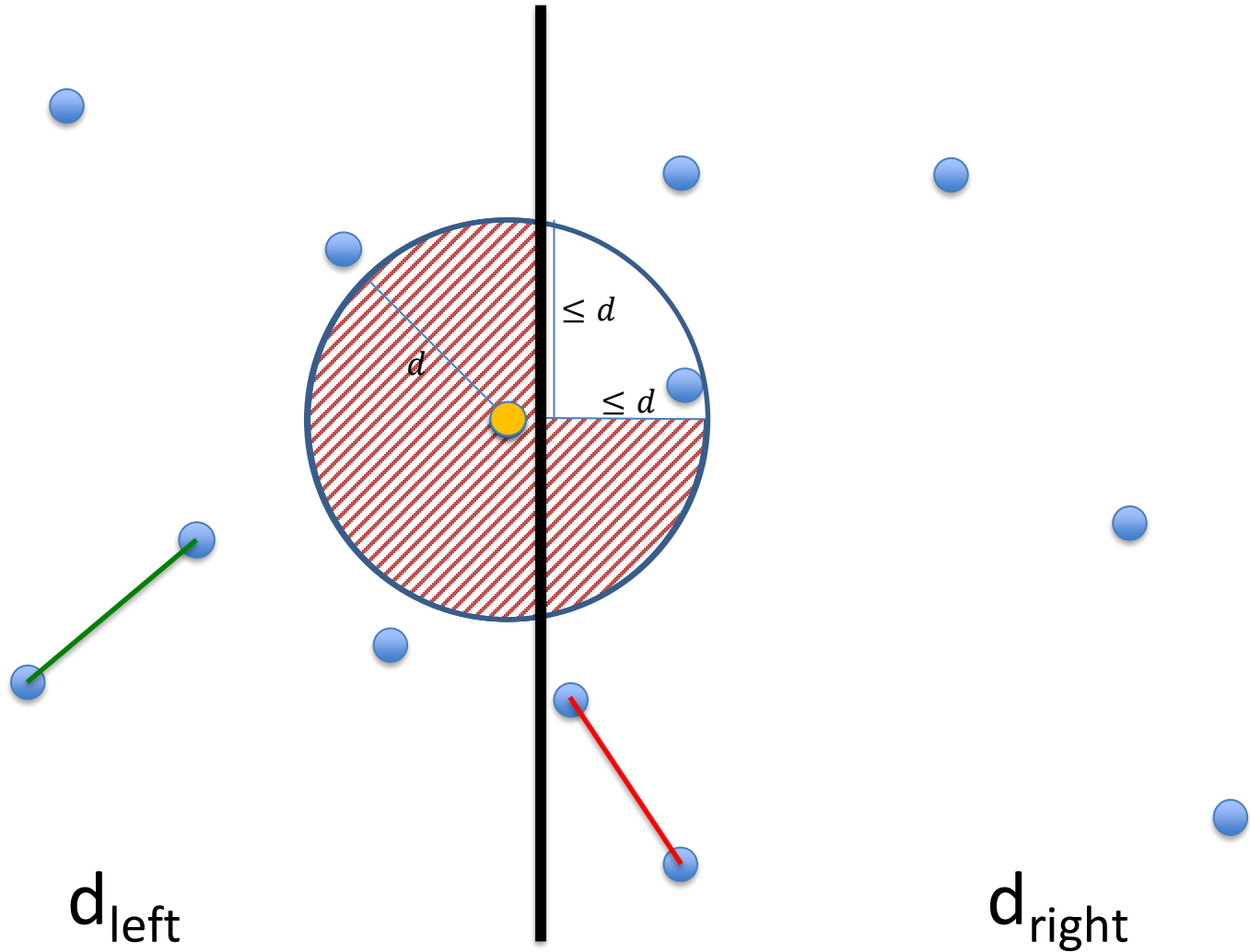
X



Done?

$$d = \min(d_{\text{left}}, d_{\text{right}})$$

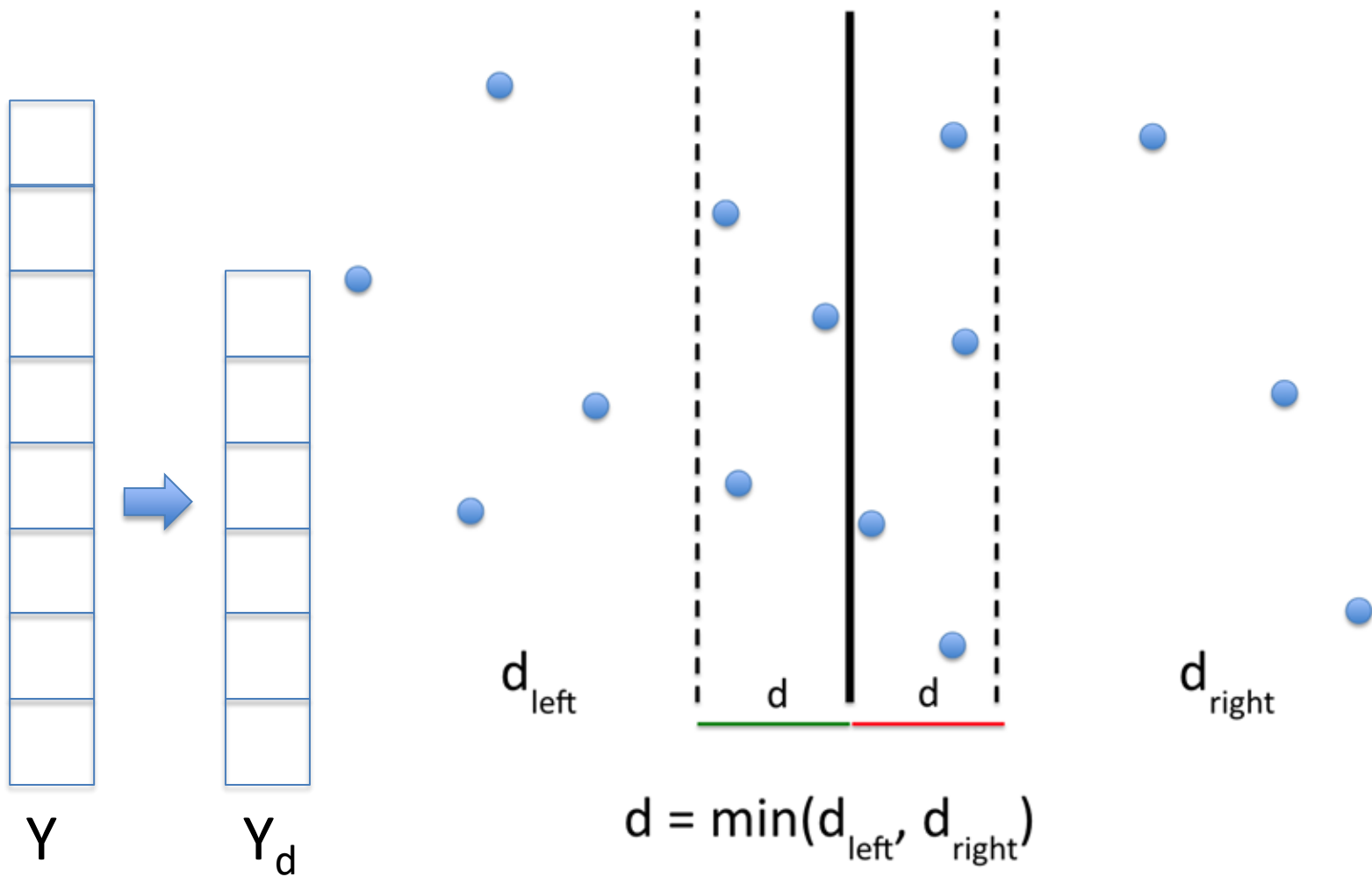




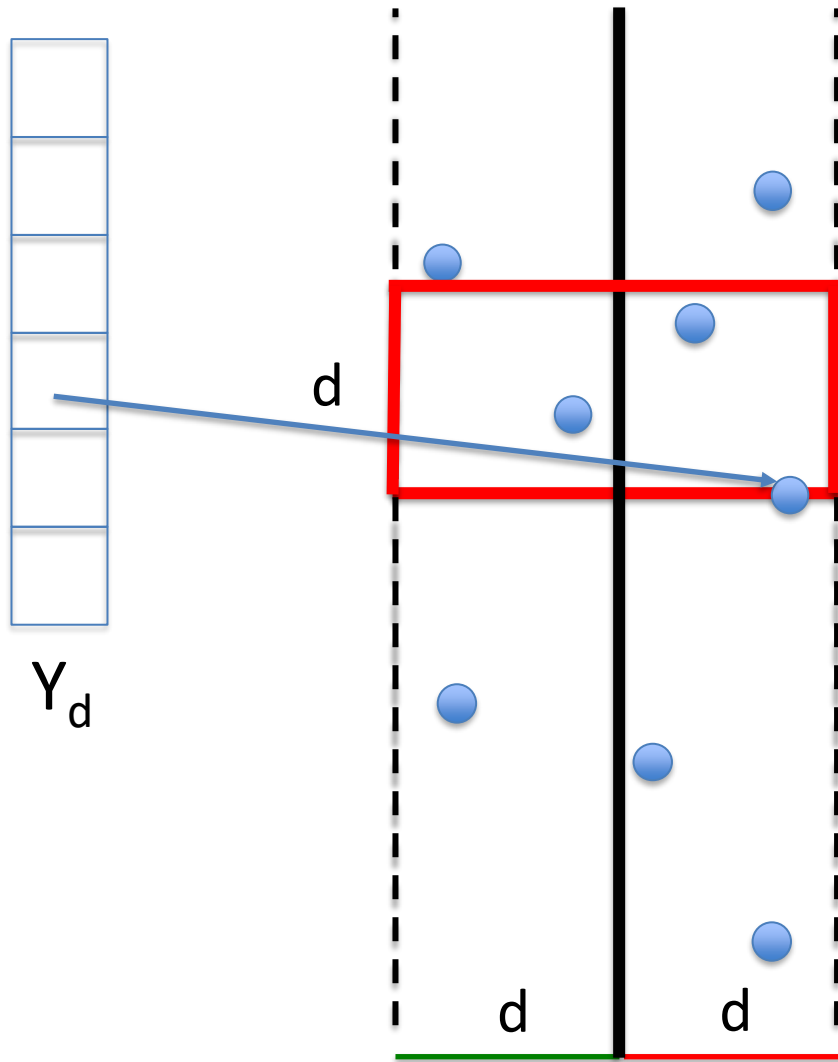
Good
news! (?)

$$d = \min(d_{\text{left}}, d_{\text{right}})$$



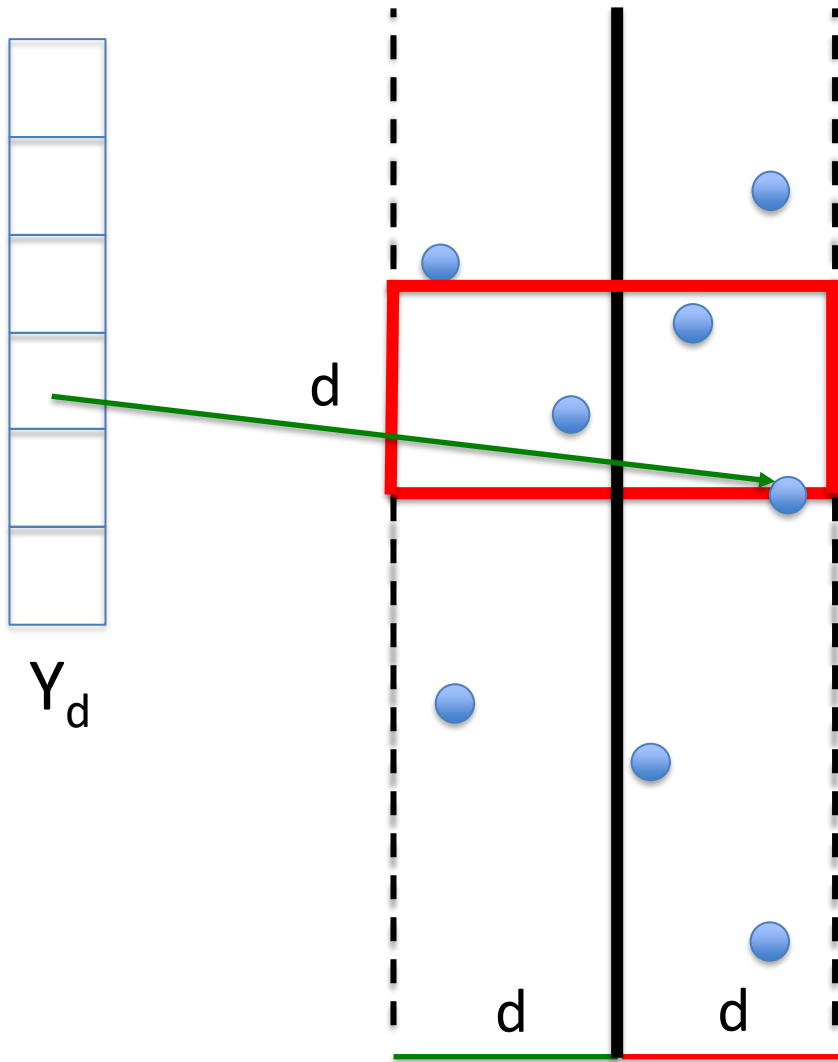


Idea: Scan through the points in the strip of width $2d$, sorted by the y-coordinate

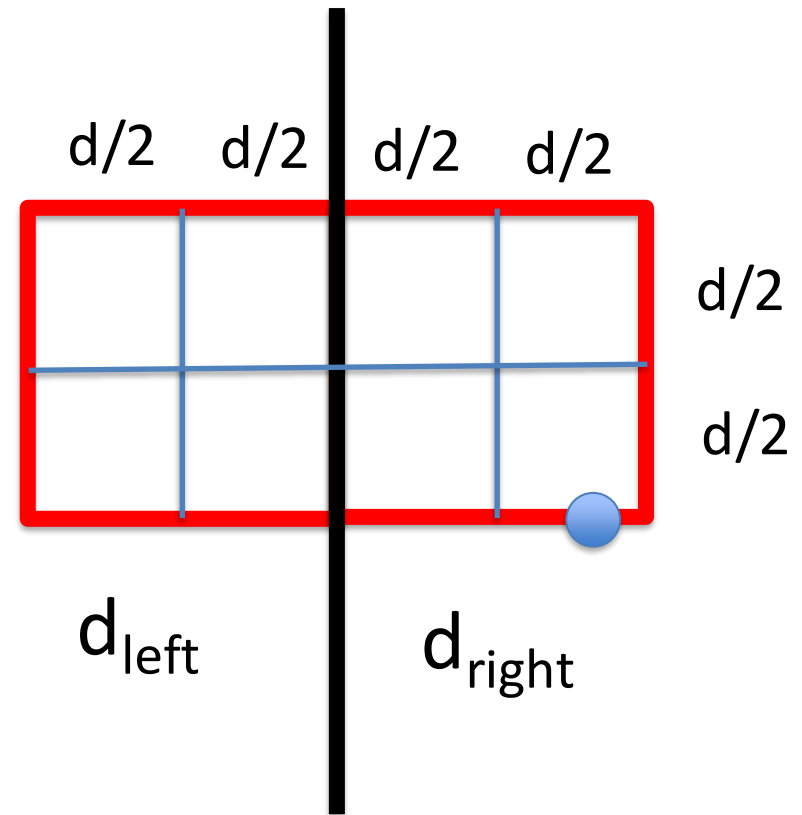


Claim: There can't be too many points in this red d -by- $2d$ rectangle? Why? How many points could there be?





Fact: There are ≤ 8 points in this red rectangle!



$$d = \min(d_{\text{left}}, d_{\text{right}})$$

Algorithm Recap

X = array of points sorted by x-coordinate

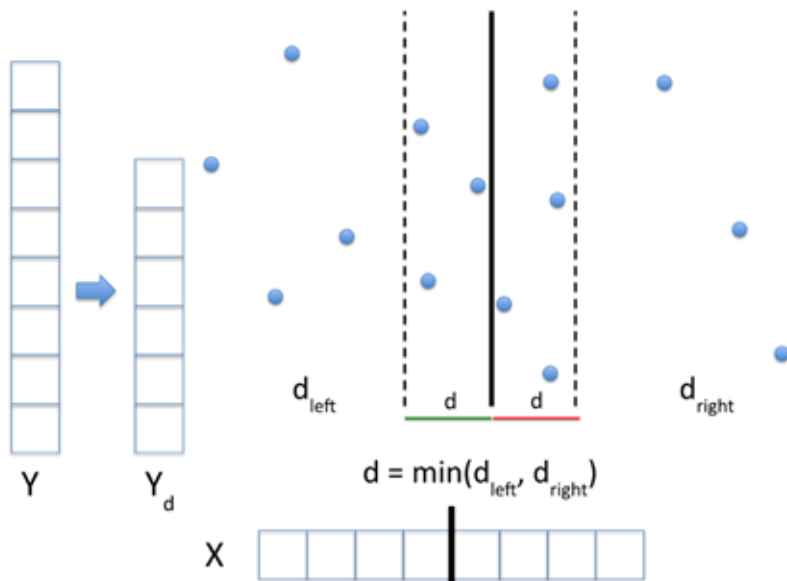
Y = array of points sorted by y-coordinate

ClosestPair(X, Y):

1. Divide points into left and right, and create $X_{\text{left}}, X_{\text{right}}, Y_{\text{left}}, Y_{\text{right}}$
2. $d_{\text{left}} = \text{ClosestPair}(X_{\text{left}}, Y_{\text{left}})$
3. $d_{\text{right}} = \text{ClosestPair}(X_{\text{right}}, Y_{\text{right}})$
4. $d = \min(d_{\text{left}}, d_{\text{right}})$
5. Y_d = array of points of distance $\leq d$ from middle line sorted by y-coord
6. For each point p in Y_d : compare p to the **next 7 points** in Y_d .
7. d_{next7} = minimum distance found in step 6.
8. Return $\min(d, d_{\text{next7}})$

Total Running Time

- Sort once by x-coordinate: $O(n \log n)$
- Sort once by y-coordinate: $O(n \log n)$
- Recursive algorithm: $T(n) = 2T(n/2) + O(n)$



Discovered in 1976 by
Jon Louis Bentley and Michael Ian Shamos

